

Quiz 7: Study guide

DSAN-6600 Deep Learning, Fall 2026

Quiz 7 is Thursday Oct 15, at the end of class. About 15 minutes, 25 points.

Closed-book. No notes of any kind, and no AI. A basic calculator is allowed. A formula sheet is handed out with the quiz, so nothing needs memorizing.

Scope: detection and segmentation

Everything in the Week 7 deck, plus Lab 6.

On the quiz:

- Transfer learning: what transfers, and the two ways to use a pretrained model
- Replacing the head, freezing, and the order the code has to go in
- The input contract: why each pretrained model has its own preprocessing
- When transfer learning stops helping
- Backbone, neck, head, and what each of the three tasks puts on the end
- Intersection over Union, including the arithmetic
- Region proposals, the R-CNN family, and non-maximum suppression
- Anchor boxes and the offsets a network predicts
- One-stage against two-stage detectors, and which to pick
- Semantic against instance segmentation, and what each one can tell you
- Upsampling: transposed convolution, the checkerboard artifact, the fixes
- Encoder and decoder, U-Net, and why the skip connections are there
- Why pixel accuracy is the wrong metric, and what to use instead
- Mask R-CNN and ROI Align
- Vision Transformers and DETR, at the level of what they do differently

On the quiz from Lab 6, which was due the day before:

- The three conditions you ran, and the order they came out in
- How many parameters a frozen ResNet-18 with a new 10-class head trains
- What the detector's own `transforms()` reported, and why
- What the score threshold does, and what NMS does
- Why one duplicate box survived per-class NMS
- What the semantic model could not tell you about the two cats

Not on the quiz: the math of attention (Week 9), focal loss beyond its name, mAP as a metric, the DETR loss in detail, and anything about training a detector from scratch, which we did not do.

Part 1: Transfer learning

What transfers

A network trained on a large dataset learns features in layers. The early layers learn edges, corners and color blobs; these are **generic**, because edges look the same in every photograph. The late layers learn whole-object parts specific to the training classes.

Transfer learning means taking a network trained on one task and reusing its learned weights on another. You keep the early layers, because they already do something you need and you do not have the data to learn it again.

The two ways to use a pretrained model

Feature extraction freezes the backbone and trains only a new head. The backbone becomes a fixed function that turns an image into a vector. Cheap, and it works with very little data.

Fine-tuning lets some or all of the backbone keep learning, at a **smaller** learning rate than you would use from scratch. More expensive, needs more data, and does better when your images are not quite like the pretraining images.

In Lab 6 the ordering was clear: fine-tuning beat the frozen backbone by about eleven points, and the frozen backbone beat training from scratch by about eleven more. Know that ordering and know why. Fine-tuning wins because the features get to adapt to your data; the frozen backbone beats scratch because 5,000 images is not enough to learn edges from nothing.

The code, and the order it goes in

```
model = resnet18(weights=ResNet18_Weights.DEFAULT)
for p in model.parameters():          # freeze everything, first
    p.requires_grad = False
model.fc = nn.Linear(model.fc.in_features, 10)  # then replace the head
optimizer = torch.optim.AdamW(model.parameters())  # then build the optimizer
[p for p in model.parameters() if p.requires_grad], lr=1e-3
```

Three things in a fixed order, and getting it wrong is silent.

Freeze first, then replace. If you replace the head and then run the freezing loop, you freeze the new head too, nothing is trainable, and `backward()` raises.

Build the optimizer **last**. An optimizer holds references to specific parameter objects. If you build it before replacing the head, it holds the old head's parameters forever; the new head gets gradients that nothing applies. Nothing raises, the trainable count looks right, and the model never learns.

`requires_grad = False` is what freezing is. It stops autograd from accumulating a gradient, so the optimizer has nothing to apply. A frozen ResNet-18 with a fresh 10-class head trains $512 \times 10 + 10 = 5,130$ parameters out of 11.2 million.

The input contract

A pretrained model expects its inputs preprocessed exactly the way they were during pretraining: the same size, the same channel order, the same scale, the same mean and standard deviation. Get it wrong and nothing raises; the accuracy is just worse than it should be.

The statistics are **not** universal. ImageNet classifiers use `[0.485, 0.456, 0.406]` and `[0.229, 0.224, 0.225]`, but the video models use different numbers, and the detectors and optical-flow models report **no** statistics at all, because they standardize internally and want plain `[0, 1]`.

So do not type the constants. Ask the weights:

```
weights = ResNet18_Weights.DEFAULT
tf = weights.transforms()          # the exact preprocessing for these weights
```

In Lab 6 you printed the detector's and saw `mean=None`, `std=None`. Feeding it ImageNet-standardized input instead drops its top detection score from 0.998 to 0.420, and nothing at all clears a 0.5 threshold.

When transfer learning does not help

Three limits, and they follow from what transfers.

A large domain gap. Edges are universal, so the early layers always transfer somewhat. But spectrograms, medical scans and satellite imagery are not photographs of the natural world, so less transfers, and they usually need fine-tuning rather than a frozen backbone.

Inputs that are not images. Reshaping tabular data into a square sometimes works and often does not. The pretrained filters are looking for spatial structure that may not be there.

A large enough dataset. With millions of labelled in-domain images, training from scratch catches up and then passes a frozen backbone. The curves cross; they just do not cross at 5,000 images.

The short version: the smaller your dataset, the more transfer learning buys you, and nearly everyone's dataset is small.

Part 2: Detection

The three tasks

Task	Output	Question it answers
Classification	one label per image	what is in this image
Detection	a box and a label per object	what, and where, and how many
Segmentation	a label per pixel	which pixels belong to what

All three use the same **backbone**, a convolutional feature extractor. What changes is the **head** bolted on the end, and sometimes a **neck** in between that combines features from several resolutions. Pretrained backbones are what make this practical: you reuse the expensive part.

Why detection is harder

A classifier produces one fixed-size output. A detector has to produce a **variable** number of outputs, because the image might hold one object or forty, and it has to localize each one. There is no fixed-size tensor that naturally represents “between zero and forty boxes,” and that awkwardness is what every detection architecture is working around.

Intersection over Union

IoU measures how much two boxes overlap:

$$\text{IoU} = \frac{\text{area of intersection}}{\text{area of union}}$$

It is 0 for boxes that do not touch and 1 for identical boxes. It is the one number you need for detection: it decides whether a prediction counts as a hit, it decides which anchors are positive during training, and it is the threshold NMS compares against.

Two things to get right in the arithmetic. The union is $\text{area}(A) + \text{area}(B) - I$, not the sum, because adding the areas counts the overlap twice. And **clamp each side of the intersection at zero**. If you do not, two boxes separated diagonally give a product of two negative sides, which is positive, and you can get an IoU of 1.000 for boxes that do not touch at all. That was the point of the `.clamp(min=0)` in Lab 6.

The region proposal idea, and R-CNN

You cannot score every possible box; there are far too many. So: propose a few thousand regions that might contain something, then classify each one. That is **R-CNN**. It used **selective search**, a non-learned algorithm that merges similar neighbouring pixels into candidate regions, then ran a CNN on each proposal separately. Accurate for its time and hopelessly slow, because 2,000 proposals meant 2,000 forward passes.

Fast R-CNN fixed the speed: run the backbone on the whole image **once**, then crop each proposal out of the resulting feature map. That is what **ROI pooling** does. One forward pass instead of 2,000.

Faster R-CNN removed selective search too, replacing it with a **region proposal network**, a small convolutional head that proposes regions from the same feature map. Now the whole thing is learned and trained end to end.

Know the progression and what each step removed. That is the exam-worthy part.

Anchors and offsets

An **anchor box** is a box of fixed size and shape at a fixed position on the feature map. The region proposal network places several at every position, typically three scales times three aspect ratios, so nine. A 50×38 feature map carries about 17,000 anchors.

The network does not predict coordinates. It predicts four **offsets** from the anchor:

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \log \frac{w}{w_a}, \quad t_h = \log \frac{h}{h_a}$$

Dividing by the anchor's width and height makes the targets **scale-free**, so the same output range works for a small object and a large one. The log on width and height keeps the predicted size **positive** whatever the network outputs. The log is natural, inverted by exp at inference.

The score threshold and NMS

A detector returns far more boxes than there are objects. Two separate filters cut them down, and they do different jobs.

The **score threshold** drops boxes the model is not confident about. In Lab 6, sweeping it over the same 18 raw detections gave 13 boxes at 0.1, 8 at 0.3, 5 at 0.5, 4 at 0.7 and 3 at 0.9. It trades recall for precision and nothing else; it cannot tell a duplicate from a real object.

Non-maximum suppression removes duplicates. Sort the surviving boxes by score, keep the highest, throw away every box whose IoU with it exceeds a threshold, and repeat on what is left. A high NMS threshold keeps near duplicates; a low one merges genuinely adjacent objects.

NMS runs per class by default. `batched_nms` takes an index per box and only suppresses within an index, so two boxes of different classes over the same object both survive. This is deliberate: a person standing in front of a car should give you both a person box and a car box.

It also means a specific failure survives. In Lab 6 there were two boxes on nearly the same region of the dog, one labelled **dog** at 0.99 and one labelled **cat** at 0.59. Their IoU is 0.836, far above the threshold, but they have different labels, so per-class NMS leaves both. Running class-agnostic `nms` over the same detections drops 5 boxes to 4 by removing the spurious cat.

One stage against two

	Two-stage	One-stage
Example	Faster R-CNN	YOLO, SSD, RetinaNet
Passes	propose, then classify	one
Speed	slower	faster, real time
Small objects	better	worse, historically
Localization	more precise	less precise

YOLO skips the proposal stage entirely. It divides the image into a grid and has every cell predict boxes and class scores directly, in one pass, so the whole thing is a single convolutional network and runs in real time.

The gap has narrowed a lot; modern one-stage detectors with a feature pyramid and focal loss are competitive on accuracy. Choose **one-stage** when you have a latency budget: video, phones, anything live. Choose **two-stage** when accuracy on small or crowded objects matters more than speed.

A **feature pyramid network** is the neck that fixed small objects. Deep feature maps know what but have low resolution; shallow ones have resolution but weak features. An FPN sends the deep features back **up** the resolution ladder and merges them with the shallow ones, so every scale gets both.

Part 3: Segmentation

Semantic against instance

Semantic segmentation labels every pixel with a class. **Instance segmentation** labels every pixel with a class *and* which object it belongs to.

The difference is what you can ask. A semantic model handed the Lab 6 photograph returns a **cat** class covering 17.2% of the pixels and a **dog** class covering 15.5%. There are two cats in that picture, and no amount of post-processing reliably recovers that: the output has no notion of object at all. Counting connected regions happens to work when the objects are separated, and fails the moment they touch. The instance model returns four separate objects with a mask each.

So: if your question involves counting, or tracking, or “which one”, you need instance segmentation.

The resolution problem

A classification backbone is built to throw away space. By the end of a ResNet a 224×224 image is a 7×7 feature map, a stride of 32. That is fine for a label and useless for per-pixel output, which needs the full resolution back. Everything in segmentation architecture is about getting it back.

Three ways to upsample

Interpolation, nearest or bilinear, has no parameters and learns nothing. **Transposed convolution** learns the upsampling. **Bilinear upsample followed by an ordinary 3×3 convolution** gets learned weights without the artifact below, and is what a lot of modern code does.

A **transposed convolution** writes a scaled copy of its kernel into the output for every input pixel, with the copies spaced by the stride, and adds the overlaps. Its output size is $s(W - 1) - 2p + k$.

The checkerboard artifact

When the **kernel size is not a multiple of the stride**, different output pixels receive different numbers of overlapping contributions, so some come out systematically brighter in a repeating grid. A constant input through an all-ones kernel 3 with stride 2 gives outputs ranging from 1 to 4 where a correct upsampler would give a constant. The pattern shows up in generated images and in masks.

Two fixes: a kernel size divisible by the stride, which is why `ConvTranspose2d(k=4, stride=2)` and `(k=2, stride=2)` are the pairs you see in real code; or bilinear upsample plus a normal convolution.

Encoder, decoder and U-Net

An **encoder** is the downsampling path that turns the image into low-resolution semantic features. A **decoder** is the upsampling path that turns those back into full-resolution output.

U-Net is an encoder and a decoder with **skip connections** between them at every resolution. The skip carries the encoder’s feature map directly across to the decoder level at the same resolution, and the decoder **concatenates** the two along the channel dimension.

Why it is needed: the deep path knows **what**, the skipped path knows **where**. The decoder receives a semantically strong but spatially vague map from below and a spatially precise but semantically weak map from across, and the next convolution learns to combine them. Without the skips, the classes are roughly right and the boundaries are mush, because boundary information was destroyed by pooling and only exists in the early layers.

A skip connection is **not** a residual connection. A residual **adds** a tensor a couple of layers later to help gradients flow through depth. A U-Net skip **concatenates** across the whole network to restore spatial detail. Same picture, different operation, different purpose.

Why accuracy is the wrong metric

Take an object covering 3.22% of an image and a model that predicts background everywhere. It scores **96.8% pixel accuracy** and a per-pixel cross entropy of 0.142, neither of which looks like failure. Its **IoU is 0.000**.

Pixel accuracy is dominated by whichever class is largest, and for segmentation that is almost always background. Report IoU or Dice instead:

$$\text{IoU} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}} \qquad \text{Dice} = \frac{2 \text{TP}}{2 \text{TP} + \text{FP} + \text{FN}}$$

A per-pixel loss has the matching problem: when the object is small, the background terms dominate the gradient. **Dice loss** is the Dice coefficient turned into a differentiable loss on probabilities, and it is scale-free in the object's size, so a small object still produces a strong signal. In practice you add the two, $\mathcal{L}_{\text{CE}} + \mathcal{L}_{\text{Dice}}$: cross entropy gives stable per-pixel gradients, Dice handles the imbalance.

Mask R-CNN and ROI Align

Mask R-CNN is Faster R-CNN with a third head. Alongside the class and the box, a small convolutional head predicts a binary mask inside each detected box. Instance segmentation for the cost of one extra head.

It needs **ROI Align** to work. ROI pooling **rounds** the projected box coordinates to the feature grid, and at a stride of 32 a rounded edge moves by up to 16 input pixels. A box regression head can learn to correct a systematic offset. A mask cannot, because the mask **is** the boundary. ROI Align keeps the coordinates as floats, places fixed sample points in each output cell, and reads the feature map at those exact positions with bilinear interpolation. No rounding anywhere, and several points of mask accuracy.

Part 4: Transformers, at preview level

You need what they do differently, not the math. Attention is Week 9.

A **Vision Transformer** cuts the image into fixed patches, typically 16×16 , flattens each one, projects it to a vector, and treats the result as a sequence.

The difference from a CNN is where spatial structure comes from. A CNN has locality and weight sharing **in the architecture** and cannot help but respect them. A ViT has neither: every patch can attend to every other patch from the first layer, so long-range relationships are available immediately, but nothing tells it that neighbouring patches are related and it has to learn that from data.

That trade shows up as a data requirement. On ImageNet-scale data alone, ViTs underperform comparable CNNs. Pretrained on much more data first, they match or beat them. So: use a pretrained ViT or fine-tune one, and do not train one from scratch on your own dataset.

DETR treats detection as **set prediction**. A fixed number of learned query vectors, typically 100, each produce one box and class directly, in parallel. No anchors, no proposals, no NMS. The cost is that it needs a **matching** step during training, assigning each prediction to at most one ground-truth box so the loss is well defined, which is done with the Hungarian algorithm.

What you should be able to do without thinking

1. Given two boxes as (x_1, y_1, x_2, y_2) , compute their IoU, with the intersection clamped at zero.
2. Given an anchor and a target box, compute t_x, t_y, t_w, t_h .
3. Write the four lines that put a new head on a pretrained backbone, in the right order, and say what breaks in each wrong order.
4. Say what preprocessing a pretrained model needs and where to get it, without typing any constants.
5. Explain what the score threshold does, what NMS does, and why they are not the same filter.
6. Say why a duplicate box can survive NMS.
7. Say what semantic segmentation cannot tell you, and what to use instead.
8. Give the transposed convolution output size, and say when it checkerboards.
9. Say what a U-Net skip connection carries and why concatenating is not adding.
10. Explain why 96.8% pixel accuracy can mean the model found nothing.
11. Say why Mask R-CNN needs ROI Align and Faster R-CNN did not.
12. Say what a ViT does not assume that a CNN does, and what that costs.

Supplemental content

Optional, for more depth on this week's material:

- Prof. Hickman's [object detection](#) and [image segmentation](#) pages
- Prof. Hickman's [transfer learning](#) page
- The `torchvision` [object detection finetuning tutorial](#)